

The missing middle

TracePcap is not short on test *infrastructure* — five CI workflows, Testcontainers, Vitest, Playwright and a committed OpenAPI snapshot are all wired and working. It is short of tests in one specific place: **the seam between components and the layer above them**. That is exactly where [#630](#) lived, and it is why six green gates let a permanently broken Download button reach production.

~71k Lines of production code, backend + frontend	466 Test cases across every suite
21.0% Backend instruction coverage (measured)	3.2% Frontend statement coverage (measured)
0 / 29 REST controllers with any test	0 / 91 React components & hooks with any test

Why #630 walked through every gate

The bug: `getDownloadUrl()` built `/api/files/...` while every backend route lives under `/api/v1`. A one-token mismatch, broken since #158, invisible until a human clicked the button. Each gate was green — and each was green *correctly*, because none of them owned the frontend ↔ backend seam.

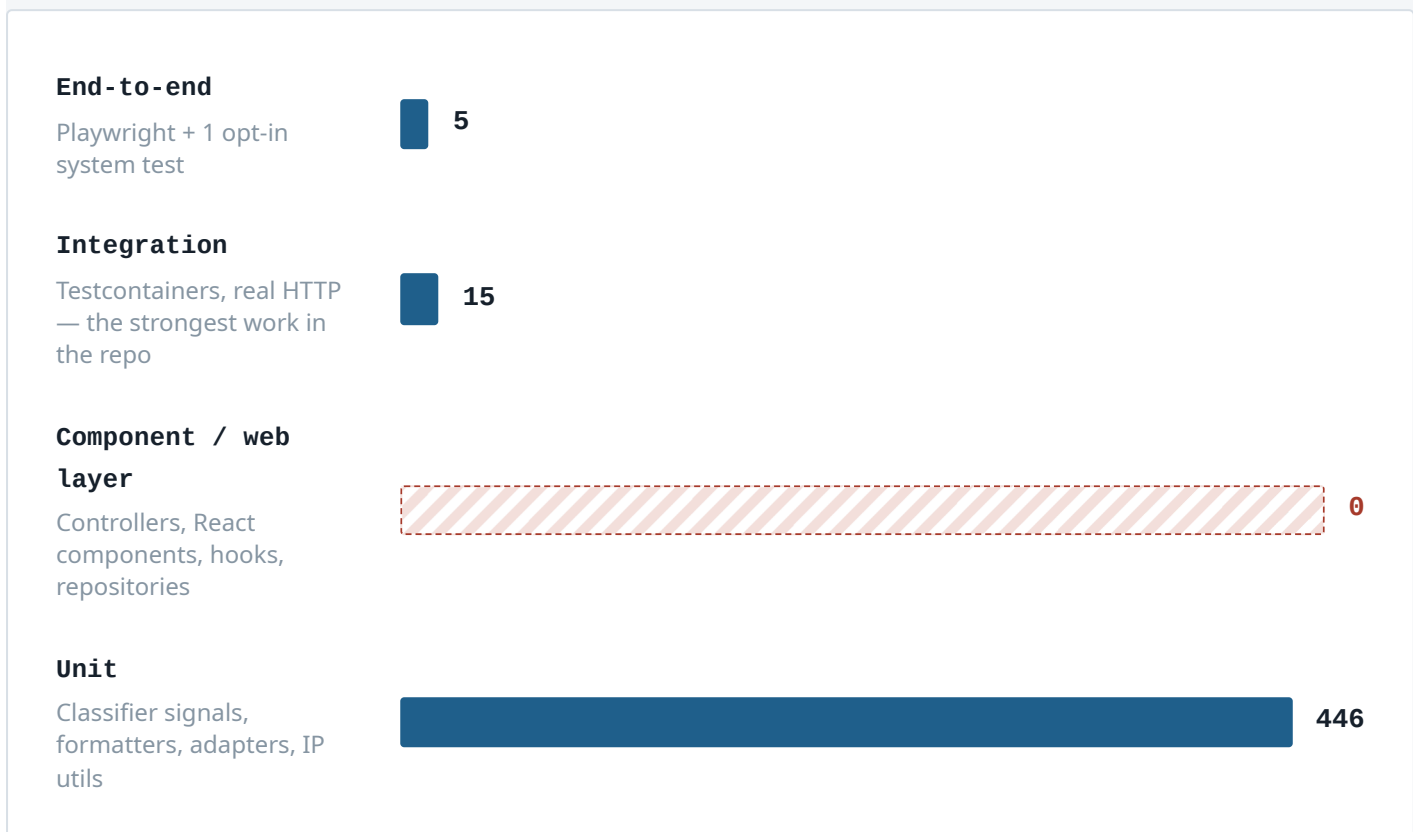
PASS test-backend	The backend was right. The route existed and worked.
PASS test-frontend	246 cases, all over pure functions. Nothing imported <code>endpoints.ts</code> .
PASS api-schema-drift	Snapshots the backend against itself. The frontend is not in scope.

PASS e2e (Playwright)	A genuinely deep demo walkthrough — that never clicks Download.
PASS system-tests	One test, and it only asserts that analysis completes.
PASS lint-frontend	Runs <code>lint:fix true</code> — it cannot fail. It never gates anything.

This is a *class* of defect, not an incident: **the frontend asserts a fact about the backend that nothing verifies**. Every hand-written URL, every assumed response shape, every enum duplicated on both sides is another instance waiting to fire. The fix shipped in #630 — `endpointPaths.test.ts` — is precisely the right instrument. It just needs to stop being a one-off.

The suite is a barbell, not a pyramid

Plotting the 466 tests by the layer they exercise shows the problem geometrically. There is weight at the bottom (pure functions) and weight at the top (full-stack), and nothing in between.



The bottom tier is real and worth keeping — the classifier signal tests and adjudicator tests are good. But it covers the code least likely to break a user, while the empty tier covers every HTTP contract, every rendered screen and every piece of state in the app.

Coverage by surface

Counted by production unit against a test that names it. No tool produced these numbers, because no tool is configured to — see Gap 1.

SURFACE	UNITS	TESTED	COVERAGE
Backend REST controllers	29	0	0%
Backend repositories	30	0	0%
React components	77	0	0%
React hooks	14	0	0%
Frontend API services	22	0	0%
Flyway migrations	40	1	3%
Backend services	52	4	8%
Frontend utils	9	6	67%

The four tested services are `TsharkEnrichmentService` , `DeviceClassifierService` , `HostIdentityService` and `CustomPrivateRangeService` . The untested 48 include the ten largest classes in the codebase — roughly **10,000 lines** in `SessionReconstructionService` (1,416), `ReportService` (1,401), `StoryService` (1,129), `FileExtractionService` (1,057) and `NetworkIntelligenceService` (875) alone.

Where the untested code actually is

With JaCoCo now wired (Phase 0, below), the backend baseline is **21.00% of instructions** — and it is not spread evenly. Ranked by package, the thinnest coverage sits squarely on the largest, most business-critical code:

PACKAGE	INSTRUCTIONS	COVERED
<code>subnets.service</code>	2,587	1.0%
<code>report</code>	6,684	2.0%

PACKAGE	INSTRUCTIONS	COVERED
<code>intelligence.service</code>	3,783	5.0%
<code>monitor.service</code>	2,885	6.3%
<code>insights.service</code>	4,891	17.5%
<code>analysis.service</code>	13,993	24.3%
<code>story.service</code>	6,309	25.9%
<code>extraction.service</code>	3,711	40.6%

`report` and `subnets.service` together are **9,271 instructions at under 2% coverage**. Reporting is the feature operators hand to other people — it is the last place a silent defect should be able to hide. The frontend baseline, **3.23% of statements**, is low enough that it is best read as "the app is untested" rather than as a percentage.

Gaps, ranked

GAP 1 • CRITICAL Nothing measures coverage

There was no JaCoCo in `pom.xml` and no `@vitest/coverage-v8` in the frontend, so every figure in the first draft of this audit had to be derived by hand from file listings. **Now closed** — both are wired, report-only, and the first honest baseline is **21.00% backend instructions / 3.23% frontend statements**. That number is the thing every other item on this list is measured against.

GAP 2 • CRITICAL The entire web layer is untested

Twenty-nine controllers, zero tests. `MockMvc`, `@WebMvcTest`, `@DataJpaTest` and `RestAssured` appear **nowhere in the repository**. Meanwhile CLAUDE.md states precise, testable API invariants — 201 on create, 204 on delete, `PagedResponse` envelope, `ErrorResponse` envelope, 1-indexed pages, DTOs never entities, `@Tag` / `@Operation` on everything. Those rules are currently enforced by code review alone.

GAP 3 · HIGH**The frontend test harness is built but unused**

`@testing-library/react`, `@testing-library/jest-dom` and `jsdom` are installed, and `vite.config.ts` already sets `environment: 'jsdom'` with a `setup.ts`. **Someone wired the entire rendering harness and no test ever used it.** The first component test costs an import, not a setup project. There is no MSW, so service-layer tests have no HTTP mock yet.

GAP 4 · HIGH**Gates that cannot fail**

`lint-frontend.yml` runs `lint:fix || true` and opens a follow-up PR; `lint-backend.yml` does the same with `spotless:apply`. Neither can ever block a merge — including the new `no-restricted-syntax` rule added in #630 to catch exactly this bug class. `system-tests` is gated behind `RUN_SYSTEM_TESTS=true`, so its single test only runs where someone remembers to set it.

GAP 5 · MEDIUM**Business logic sits in classes too large to test**

The correlation is not accidental: the untested services are the big ones. A 1,400-line service with database, filesystem and subprocess dependencies threaded through it resists unit testing, so it never gets any, so it keeps growing. The [five-stage architecture](#) (Ingest → Extract → Scan → Adjudicate → Narrate) already defines the seams to split them along, and ArchUnit already enforces those seams.

GAP 6 · MEDIUM**E2E breadth is one path deep**

Four Playwright tests total. The demo walkthrough is genuinely thorough down one happy path, but **most of the twenty routes are never loaded by any test**, and no test exercises a download or export button anywhere in the app — the precise surface #630 broke.

Roadmap

Ordered by return per hour, not by tidiness. Phases 0–2 close the #630 bug class outright and are worth doing before anything else; 3–5 are the long tail.

PHASE 0

Turn on the lights done — #659

- Add the **JaCoCo** plugin to `backend/pom.xml` and `@vitest/coverage-v8` to the frontend.
- Publish both reports as PR artifacts. **Report-only at first** — a hard threshold on day one just gets bypassed.
- Once a baseline exists, add a **ratchet**: coverage may not fall below the committed number. This is far more effective than a target nobody hits.

PHASE 1

Close the seam #630 came through ~2-3 days

- **Generate the frontend client from** `openapi/baseline.json` with `openapi-typescript`. This is the highest-value item in the audit: it converts the whole bug class from a runtime 404 into a *compile error*. The baseline is already committed and already enforced in CI — the generator is the missing half.
- Until then, extend `endpointPaths.test.ts` to assert **method + path pairs**, not paths alone, so a GET pointed at a POST-only route also fails.
- **Delete the five dead endpoints** the test exempted (`FIVE_WS` , `KILL_CHAIN` , `TIMELINE_DATA` , `TIMELINE_RANGE` , `SNAPSHOT_BASELINE`). Exemptions age into permanent blind spots.
- Split lint into **a blocking check job and the existing auto-fix job**, so the rule added in #630 can actually stop a merge.

PHASE 2

Test the web layer as a contract ~1 week

- **Start with one ArchUnit rule set, not 29 test classes.** You already have `LayerDependencyTest` with four frozen rules. Add rules asserting that every `@RestController` declares a version-agnostic plural kebab-case path, returns a DTO rather than a JPA entity, and carries `@Tag / @Operation`. One file covers all 29 controllers *and every controller anyone adds later*.
- Then add `@WebMvcTest` slices for the controllers that carry real logic — validation, pagination, permissions — rather than all 29 uniformly.
- Grow `PipelineIntegrationTest`, which is already the best-designed suite in the repo, to cover **file download and export** end to end.

PHASE 3 **Wake the frontend harness up** ~1 week

- Add **MSW** and test the 22 API services against mocked HTTP — error paths, retries, envelope unwrapping.
- Then the **14 hooks**: highest logic density, no DOM complexity, best value per test in the whole frontend.
- Then roughly ten high-traffic components. **Do not attempt all 77** — target the ones with conditional rendering and state, not presentational leaves.

PHASE 4 **Break up what can't be tested** ongoing

- Before refactoring any of the top-five services, write **characterisation tests** that pin current behaviour — including behaviour that looks wrong. Refactoring 1,400 untested lines without them trades one risk for a larger one.
- Extract pure decision logic (scoring, adjudication, thresholds) out of the I/O-bound services. The `hostclassification` signal tests show this already works well here.

PHASE 5 **Broaden the top of the pyramid** ~3 days

- Add a **parameterised route-sweep test**: visit all 20 routes, fail on any console error or failed network request. Cheap, and it would have caught #630 directly.
- Cover **every download and export control** — the untested affordance class.
- Run `system-tests` on merge to `main` rather than behind an opt-in environment variable.

Two process changes worth more than any single test

Every bug fix ships with a test that fails without the fix. #630 did this — the PR verified that restoring the old prefix broke 87 assertions. That instinct was correct and it should be the standing rule, not a one-off. It converts each production defect into permanent coverage of exactly the code that has already proven it can break.

The PR template asks one question: which test covers this? "Manually verified" is a valid answer for a spike and a warning sign on a feature. Making the answer visible in every PR does more to change the coverage trend than any threshold.

Audit performed against `feature/623-native-tool-calling` @ `40b2d9d` · Counts derived from source inventory: 339 backend classes / 236 frontend modules / 33 backend test files / 9 frontend test files / 3 Playwright specs · 13 GitHub Actions workflows reviewed